Kiro Workshop

Agenda

Time	Topic
9:30	Introductions
9:45	The Challenge
10:00	The Renaissance Developer
10:15	Enter Kiro + Demo
11:00	☕ Break
11:15	**Guided Workshop (Part 1)**
12:00	🍽️ Lunch
13:00	**Guided Workshop (Part 2)**
14:30	Portbase Guidelines + Q&A
15:00	End

Introduce Yourself

One challenge you've faced with AI coding tools

Workshop Goal

Build a Notes Webapp using spec-driven development

Guided Workshop — we build together, step by step

- Understand the spec-driven workflow (Requirements → Design → Tasks)
- Use Kiro to generate and implement specifications
- Experience AI-assisted development with human oversight
- Deploy a working application by end of day

AI is Changing Software

2024
Assistants

Helping developers write code faster

2025
Agents

Completing development tasks end-to-end with human in the loop

2026
Autonomy

Completing development tasks end-to-end with bounded independence

Example: 2024 Assistants

Task: "Write a login function"

Flow:

- Developer asks AI for code
- AI suggests code snippet
- Developer manually integrates snippet
- Developer writes tests
- Developer updates documentation
- Developer handles error cases

Result: *Human does most work, AI helps with snippets*

Example: 2025 Agents

Task: "Add JWT authentication with refresh tokens"

Flow:

- AI generates requirements → **Human approves**
- AI designs architecture → **Human approves**
- AI implements complete feature (handlers, validation, tests, docs)
- AI runs tests and reports results
- **Human reviews & merges**

Result: *Agent does end-to-end work, human controls checkpoints*

Example: 2026 Autonomy

Task: "Users report slow login"

Flow:

- AI autonomously investigates performance issue
- AI proposes optimization approach
- AI implements fix across multiple files
- AI runs full test suite
- AI creates pull request with detailed explanation
- **Human reviews final PR**

Result: *AI works independently with minimal oversight*

The Promise of Agentic Development

Autonomy

Agents can take on and complete increasingly challenging tasks autonomously

True Collaboration

Developers and agents work together to get more done

Higher Quality

Agents handle the heavy lifting of increasing test coverage, documenting code, optimizing performance, fixing security vulnerabilities

Developers Stay Responsible

AI Assists, You Decide

AI generates suggestions and implementations — humans review, approve, and own the final code

Accountability Remains

You're responsible for security, compliance, and correctness. AI is a powerful tool, not a replacement for engineering judgment

Human in the Loop

Every checkpoint requires human approval. Specs, design, and implementation all go through your review

Challenges with AI Development

Scaling AI Development

AI coding tools excel at small tasks but can fail with complex projects

Limited Control

Existing tools make it difficult to collaborate with and manage agents

Code Quality

Getting a project from proof-of-concept to production while maintaining quality control becomes increasingly difficult

The Renaissance Developer

Werner Vogels, AWS re:Invent 2025

AI won't replace developers who evolve. The Renaissance Developer combines five qualities: **curiosity**, **systems thinking**, **precise communication**, **ownership** of code (not blind "vibe coding"), and **T-shaped expertise**.

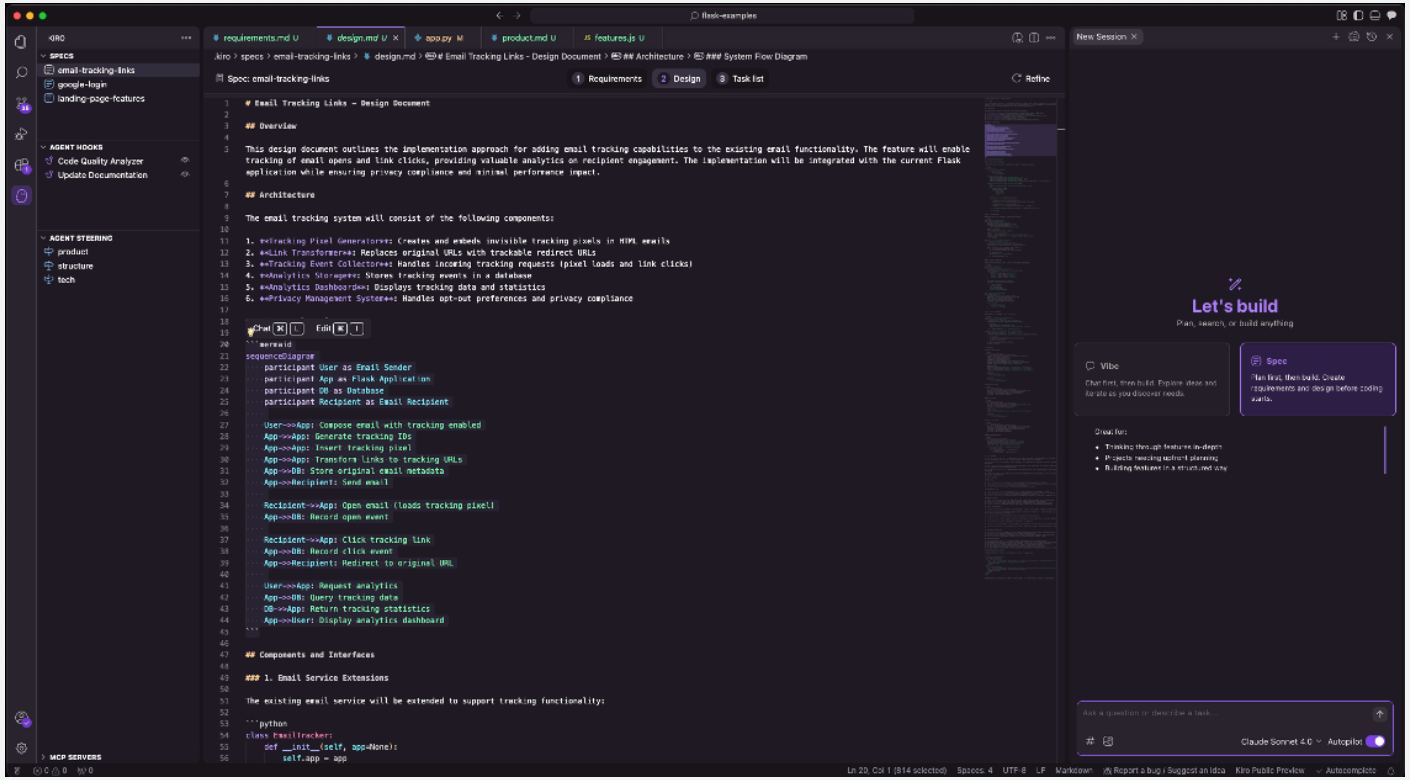
Master your tools. Own your output. Keep learning.

Enter Kiro

Spec-Driven Development for Agentic AI Coding

💡 Idea → 📋 Requirements → 🏗️ Design → ✅ Tasks → 💻 Code → 🚀 Production

The AI IDE


requirements.md
design.md
app.py
product.md
features.js
SPECS
email-tracking-links
google-login
landing-page-features
AGENT HOOKS
Code Quality Analyzer
Update Documentation
AGENT STEERING
product
structure
tech
MCP SERVERS
Spec: email-tracking-links
1 Requirements
2 Design
3 Task list
Refine
Email Tracking Links - Design Document
Overview
This design document outlines the implementation approach for adding email tracking capabilities to the existing email functionality. The feature will enable tracking of email opens and link clicks, providing valuable analytics on recipient engagement. The implementation will be integrated with the current Flask application while ensuring privacy compliance and minimal performance impact.
Architecture
The email tracking system will consist of the following components:
1. **Tracking Pixel Generator**: Creates and embeds invisible tracking pixels in HTML emails
2. **Link Transformer**: Replaces original URLs with trackable redirect URLs
3. **Tracking Event Collector**: Handles incoming tracking requests (pixel loads and link clicks)
4. **Analytics Storage**: Stores tracking events in a database
5. **Analytics Dashboard**: Displays tracking data and statistics
6. **Privacy Management System**: Handles opt-out preferences and privacy compliance
Chat
Edit
```mermaid
sequenceDiagram
participant User as Email Sender
participant App as Flask Application
participant DB as Database
participant Recipient as Email Recipient
User->>App: Compose email with tracking enabled
App->>App: Generate tracking IDs
App->>App: Insert tracking pixel
App->>App: Transform links to tracking URLs
App->>DB: Store original email metadata
App->>Recipient: Send email
Recipient->>App: Open email (loads tracking pixel)
App->>DB: Record open event
Recipient->>App: Click tracking link
App->>DB: Record click event
App->>Recipient: Redirect to original URL
User->>App: Request analytics
App->>DB: Query tracking data
DB->>App: Return tracking statistics
App->>User: Display analytics dashboard
```
Components and Interfaces
1. Email Service Extensions
The existing email service will be extended to support tracking functionality:
```python
class EmailTracker:
def __init__(self, app=None):
self.app = app
New Session
Let's build
Plan, search, or build anything
Vibe
Spec
Claude Sonnet 4.0
Autopilot
```

Spec driven development

Kiro turns your prompt into clear requirements, system design, and discrete tasks

Iterate with Kiro on your spec and architecture

Kiro agents implement the spec while keeping you in control

Agent Hooks

Delegate tasks to AI agents that trigger on events such as 'file save'

Agents autonomously execute in the background based on your pre-defined prompts

Agent hooks help you scale your work by generating documentation, unit tests, or optimizing code performance

Advanced Context Management

Connect to docs, databases, APIs, and more with native MCP integration

Configure how you want Kiro agents to interact with each project via steering files

Drop an image of your UI design, or a photo of your architecture whiteboarding session, and Kiro can use it to guide its implementation

Timeline Checkpointing

Rollback to existing points in the execution log

Safety net to safely explore multiple approaches to a problem

Snapshots the contents of each modified file and then restores that snapshot when reverting to the checkpoint

Claude Models in Kiro

Model	Best For
Auto	Intelligent routing (default)
Haiku 4.5	Fast, cost-effective tasks
Sonnet 4.0 / 4.5	Daily coding, speed + quality
Opus 4.5	Complex architecture, deep reasoning

Reducing Credit Usage

Be Specific

Detailed steering files and clear specs = correct code first time

Edit, Don't Regenerate

Fix typos or small issues manually instead of asking Kiro to rewrite entire files

Small Tasks

Atomic tasks give predictable results with fewer retries

Context upfront = fewer iterations = fewer credits

Spec-Driven Development

```
1. Requirements (WHAT)  → User stories + acceptance criteria
2. Design (HOW)         → Architecture + data models + APIs
3. Tasks (DO)           → Ordered implementation checklist
```

Key principle: No code without documented specification

Specs

The Three Core Files

```
.kiro/specs/feature-name/
├── requirements.md   ← WHAT to build
├── design.md         ← HOW to build it
└── tasks.md          ← Implementation steps
```

requirements.md

```
## User Story: Personalized Greeting

As a user, I want to receive a personalized greeting
so that I feel welcomed when using the application.

### Acceptance Criteria

WHEN a name is provided
THE SYSTEM SHALL return a greeting containing that name
```

EARS = Easy Approach to Requirements Syntax

A structured syntax for writing requirements that can be validated and tested.

EARS Patterns

Pattern	Template
Universal	THE SYSTEM SHALL [action]
Event-Driven	WHEN [event] THE SYSTEM SHALL [action]
State-Driven	WHILE [state] THE SYSTEM SHALL [action]
Unwanted	IF [condition] THEN THE SYSTEM SHALL [action]
Optional	WHERE [feature] THE SYSTEM SHALL [action]

design.md

Technical Architecture

```
Request ──▶ Validation ──▶ Greeting Generator
                                  │
                          ┌───────┴───────┐
                          ▼               ▼
                        Formal          Casual
```

Design docs include: architecture, data models, API definitions, schemas

tasks.md

Implementation Checklist

```
- [x] 1. Set up project structure
- [x] 2. Define input schema with Zod
- [ ] 3. Implement greeting handler
      - Validate name input
      - Generate greeting by style
      - Handle errors
      _Requirements: US-1.1, US-1.2_
```

Click a task → Kiro implements it with full context.

Steering Docs

Project Context Files

```
.kiro/steering/
├── product.md    ← WHAT the product is
├── tech.md       ← HOW we build (stack, patterns)
└── structure.md  ← WHERE code lives
```

Loaded into every conversation = consistent context

product.md

```markdown
# Product Context

## Overview
Device telemetry ingestion service for IoT fleet management.

## Core Features
- Real-time telemetry ingestion from IoT devices
- Data validation and enrichment
- Alerting on anomalous readings
```

tech.md

```md
# Technical Standards

## Stack
- Runtime: Node.js 20.x, TypeScript 5.x
- Cloud: AWS Lambda, DynamoDB, IoT Core
- Validation: Zod schemas

## Patterns
- AWS SDK v3 modular imports
- Structured JSON logging
```

structure.md

```
# Project Structure

src/
├── handlers/     # Lambda entry points
├── services/     # Business logic
├── models/       # TypeScript types + Zod schemas
└── utils/        # Shared utilities

infrastructure/   # AWS CDK definitions
```

MCP Servers

Model Context Protocol

MCP connects Kiro to external tools and data sources:

```
Kiro ◀──▶ MCP Server ◀──▶ AWS / GitHub / Docs
```

MCP servers connect Kiro to external APIs and data sources.

Examples of MCP Servers

AWS
Search AWS documentation

Context7
TypeScript, CDK best practices

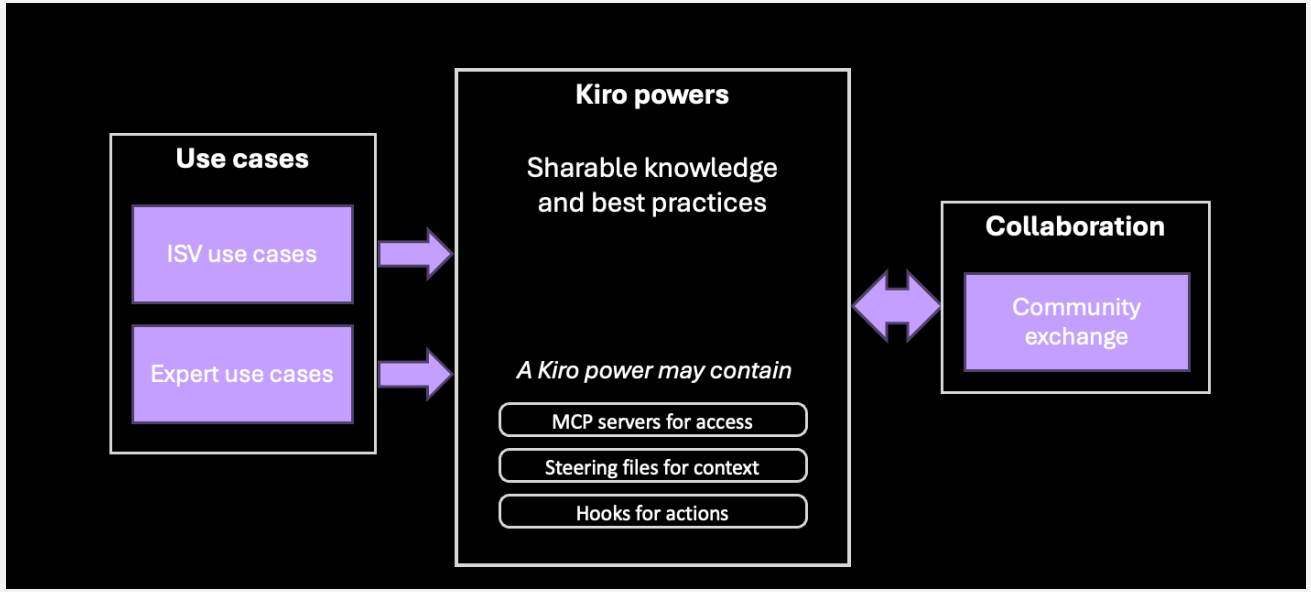
GitHub
Access repos, issues, PRs

PostgreSQL
Query database schemas

Java
Maven, Spring Boot patterns

Kiro Powers

Extend Kiro agent capabilities through sharable best practices



Kiro powers – Benefits

Easy to Use

- One-click install in Kiro
- Loads only what's needed
- Customize to fit your project

Expert Knowledge

- Best practices from tool vendors
- Curated guidance from experts
- Share your own powers with the community

Full Workflow

- Frontend and backend development
- Build and deploy automation
- Performance and security monitoring

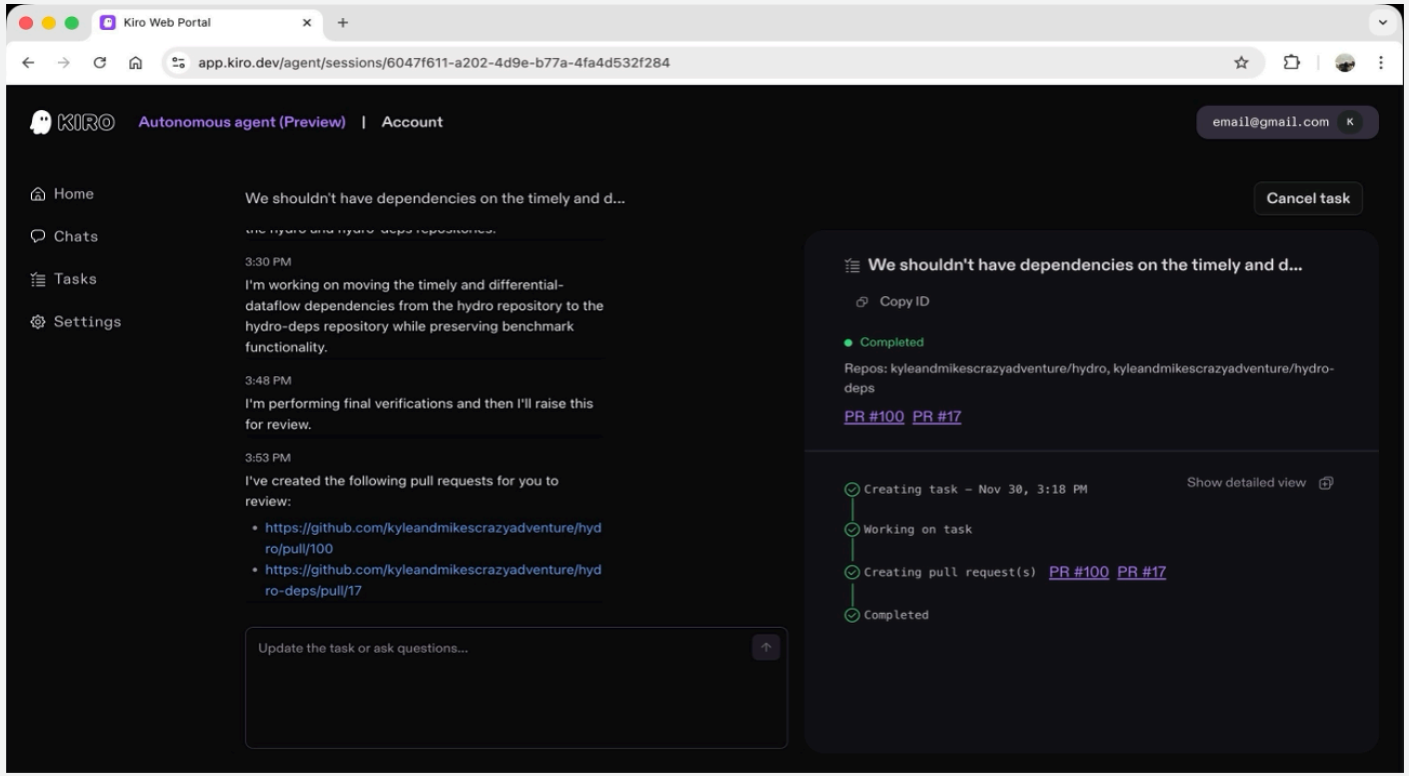
Kiro Power Example

Think of them as expert toolkits.

For example, if there's an **AWS Infrastructure power** installed,
I can use it to:

- Access AWS documentation and best practices
- Validate CloudFormation templates
- Check security compliance
- Troubleshoot deployments
- Generate architecture diagrams

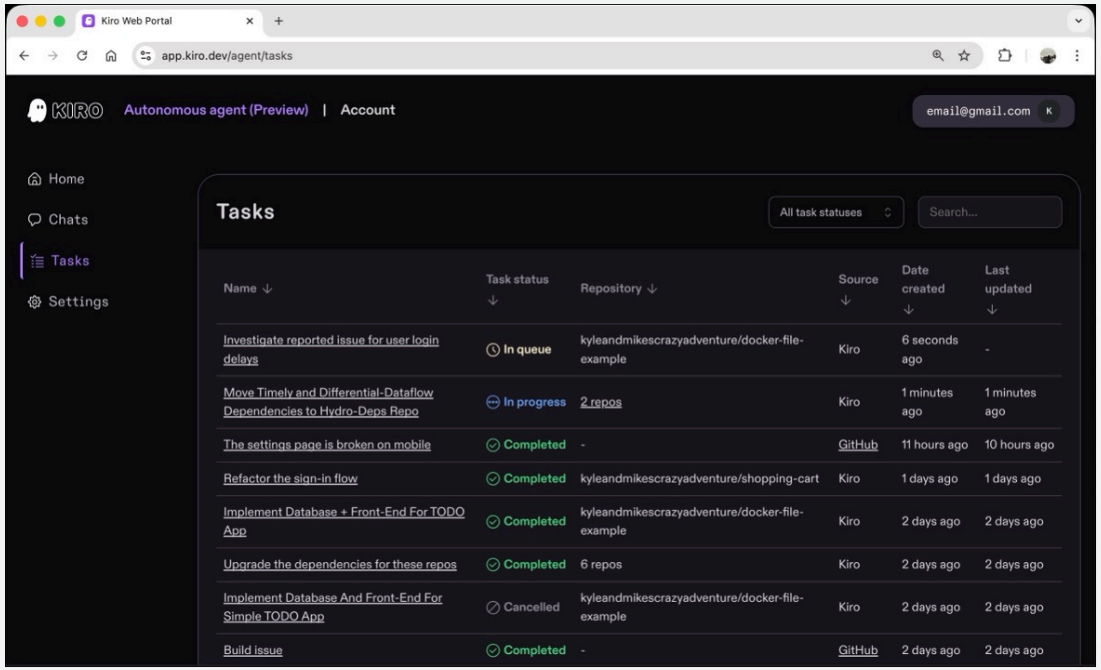
Kiro autonomous agent



The autonomous agent that extends your flow in Kiro, now available in gated preview for both individuals and teams.

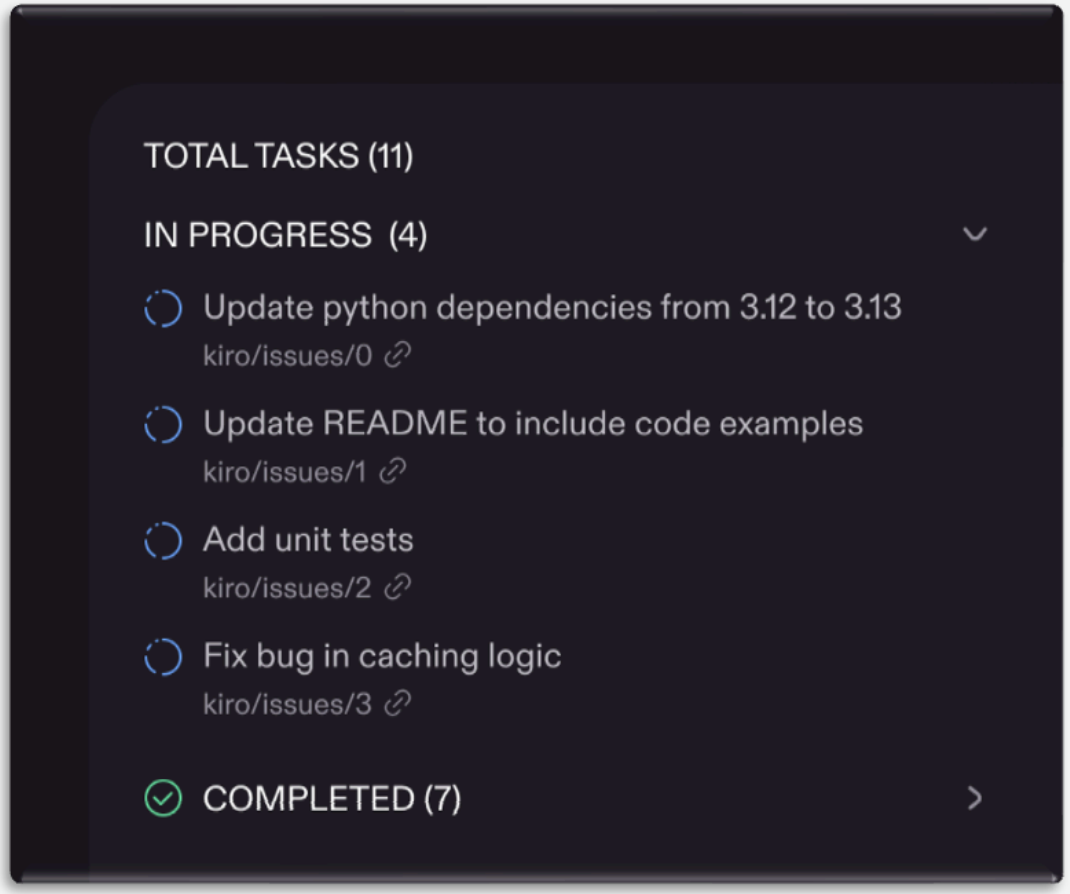
Parallel execution without blocking your flow

Chat with it, describe a change you need, or an improvement you want, and execute up to 10 tasks concurrently. It independently figure out how to get the work done.



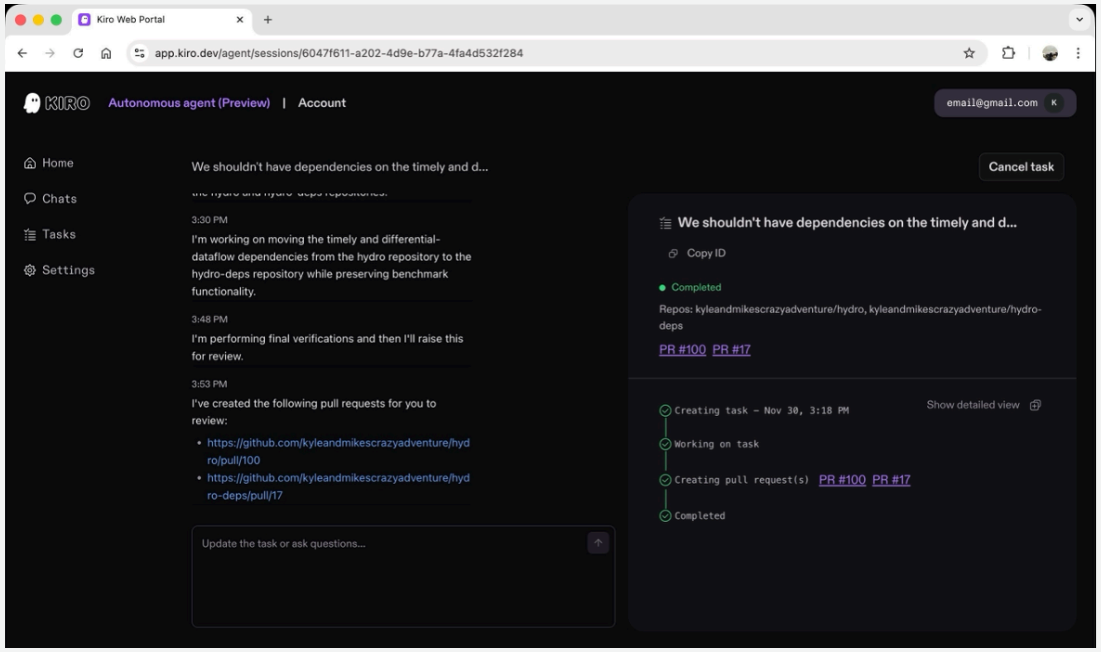
Context that stays with your work

- Kiro autonomous agent is always there and maintains context across your work.
- Kiro remembers feedback and applies that pattern to subsequent change.
- When Kiro encounters similar architectural decisions, it considers existing implementations and preferences.
- You're not re-explaining your codebase or repeating the same work—Kiro already knows how you work and gets better with each interaction.



Multi-repo changes in one move

Describe the job once. Kiro treats the multi-repo work as a unified task—identifying the affected repo, analyzing how each service uses the library, updating code following your patterns, running full test suites, and opens pull-requests for review, all while you work on something else.



For Java Developers

Your Patterns Matter

AI that ignores your DI framework, package structure, or logging conventions isn't useful. Steering files teach Kiro YOUR architecture before any code is generated.

Traceability

Specs create auditable chains: Requirement → Design → Task → Code. When compliance asks "why?", you have the documentation. Try getting that from ChatGPT.

IDE Complement

IntelliJ has superior refactoring. Kiro adds a spec layer that doesn't exist elsewhere. Many teams use both: IntelliJ for coding, Kiro for AI-assisted feature development.

Java-Aware AI Development

📦 **Package Structure**
structure.md defines your conventions

🌱 **Spring Patterns**
DI approach, @Service/@Repository usage

📝 **Logging**
SLF4J/Logback, not System.out

⚠️ **Exceptions**
Custom hierarchy respected

🔧 **Build Tool**
Maven/Gradle conventions

🧪 **Testing**
JUnit 5, Mockito patterns

The Java Engineer's AI Problem

```
You: "Create a UserService"

ChatGPT:                                  What you actually needed:
──────────────────────────────────        ──────────────────────────────────
public class UserService {                public interface UserService { }
  @Autowired
  private UserRepository repo;            @Service
                                          public class UserServiceImpl
  public User getUser(Long id) {              implements UserService {
    return repo.findById(id);
  }                                         private final UserRepository repo;
}
                                            public UserServiceImpl(UserRepository r) {
                                              this.repo = r;
                                            }
                                          }
```

Field injection vs constructor injection. No interface.

Java Steering Example

```
# tech.md – Java/Spring Standards

## Framework
- Java 21, Spring Boot 3.x, Constructor injection ONLY

## Architecture
- Interface + Impl pattern, Package: com.portbase.{domain}.{layer}

## Conventions
- SLF4J with @Slf4j, Custom exceptions extend BaseException
- DTOs separate from entities (MapStruct), @Valid for validation
```

Before and After: Java Code Generation

❌ Without Steering

```java
@Service
public class OrderService {
    @Autowired
    private OrderRepository orderRepo;

    public Order create(Order order) {
        System.out.println("Creating order");
        return orderRepo.save(order);
    }
}
```

✅ With Steering

```java
@Service
@Slf4j
public class OrderServiceImpl implements OrderService {
    private final OrderRepository orderRepo;

    public OrderServiceImpl(OrderRepository orderRepo) {
        this.orderRepo = orderRepo;
    }

    public Order create(CreateOrderRequest req) {
        log.info("Creating order: {}", req.getOrderId());
        return orderRepo.save(mapper.toEntity(req));
    }
}
```

Java Developer Workflow in Kiro

1
Setup
product.md, tech.md, structure.md

→

2
Spec
requirements.md, design.md, tasks.md

→

3
Review
Check requirements

→

4
Design
Review architecture

→

5
Implement
Click tasks

→

6
Validate
Hooks check

Java MCP Server

Spring Boot

- Auto-configuration patterns
- Actuator endpoints
- Security configuration
- Data JPA conventions

Maven/Gradle

- Dependency management
- Plugin configuration
- Multi-module setup
- Build lifecycle

Testing

- JUnit 5 patterns
- Mockito best practices
- @SpringBootTest setup
- Testcontainers integration

Real-time Java documentation access

Quick Recap

Specs

Requirements → Design → Tasks

Steering

Project context for consistent AI output

Hooks

Automated quality checks on save

MCP

Connect to external tools and docs

Recently Added to Kiro

🔌 ACP Support
Agent Client Protocol — like LSP but for AI agents. Use Kiro in JetBrains, Eclipse, Neovim, Emacs, Zed.
Feb 2026

🏗️ Design-First Specs
Start a feature spec from technical architecture instead of requirements. Ideal for porting existing systems.
Feb 2026

🐛 Bugfix Workflow
Structured debugging: describe the issue → root cause analysis → fix design → regression prevention.
Feb 2026

🤖 Autonomous Agent
Background agent that runs tasks independently. Up to 10 parallel tasks, works across multiple repos.
Dec 2025 (Preview)

Kiro + IntelliJ

Use Both — They're Complementary

IntelliJ excels at: refactoring, debugging, Maven/Gradle integration, code navigation, Spring support

Kiro adds: spec-driven development, AI-assisted feature generation, steering files, automated hooks

Via **ACP** (Agent Client Protocol), Kiro CLI works inside IntelliJ. Keep your IDE, add Kiro's AI capabilities.

Setup in IntelliJ (2 min)

1. Install Kiro CLI:
   `curl -fsSL https://cli.kiro.dev/install | bash`
2. Open AI Chat tool window
3. Click menu → "Add Custom Agent"
4. Configure `acp.json` with Kiro agent
5. Select "Kiro Agent" from dropdown

Guide: kiro.dev/blog/kiro-adopts-acp

Java Engineer Workflow: Kiro IDE + IntelliJ

🟣 Kiro IDE

- Create & review specs (requirements → design → tasks)
- Set up steering files (tech.md, product.md)
- Configure hooks for automation
- Generate new features/modules
- Use for AI-heavy feature development

🟠 IntelliJ + Kiro CLI

- Day-to-day Java coding
- Refactoring (IntelliJ is superior)
- Debugging with IntelliJ tools
- Maven/Gradle builds
- AI assistance via ACP when needed

Both read the same `.kiro/` folder — steering files, specs, and hooks work in both environments.
Configure once, use everywhere.

Java Engineer with Kiro

🌅 Morning: Plan & Design

🟣 Kiro IDE

- "Add payment processing feature"
- Review generated requirements.md
- Approve design.md architecture
- Check tasks.md implementation plan

☀️ Afternoon: Implement

🟠 IntelliJ + Kiro CLI

- Open PaymentService.java
- Ask Kiro: "Implement task 3"
- Use IntelliJ refactoring tools
- Debug with IntelliJ debugger
- Run Maven tests

📁 Shared: .kiro/ folder

steering/tech.md (Spring conventions) • **specs/** (requirements, design, tasks) • **hooks/** (automation)
→ Same AI context in both environments

Let's start using Kiro

Guided Workshop

Building a Notes Webapp

What We'll Build

📝 Create Notes

Add new notes with title and content

📋 List Notes

View all your saved notes

✏️ Edit Notes

Update existing notes

🗑️ Delete Notes

Remove notes you no longer need

We'll build this step-by-step using Kiro's spec-driven workflow

Workshop Resources

Workshop link:

https://catalog.us-east-1.prod.workshops.aws/join?access-code=b815-081e28-30

Stay Up to Date

AWS News

www.aws-news.com
AWS announcements and service updates

Kiro Updates

kiro.dev/docs
Documentation and new features

AWS Blogs

aws.amazon.com/blogs
Deep dives and best practices

Data Privacy & Model Training

Sign-in Method	Data Used for Training?	Data Storage
Social login (GitHub, Google) or **AWS Builder ID**	✅ Yes (opt-out available)	US East (N. Virginia)
AWS IAM Identity Center (Pro/Pro+/Power/Enterprise)	❌ No	Not stored

How to Opt Out of Model Training (Free/Individual Users)

Settings (Cmd/Ctrl+,) → User → Application → Telemetry and Content
Uncheck: **"Content Collection for Service Improvement"** ← this is the one that affects model training

Dos and Donts at Portbase